

JIMSKY STUDIO

You talk to it, it makes things, and what it makes shows up on a stage you own — from a phone, a tablet, or a kitchen counter. This is what is actually built, what it runs on, and what is still missing.

10

VOICES AS ENTITIES

629

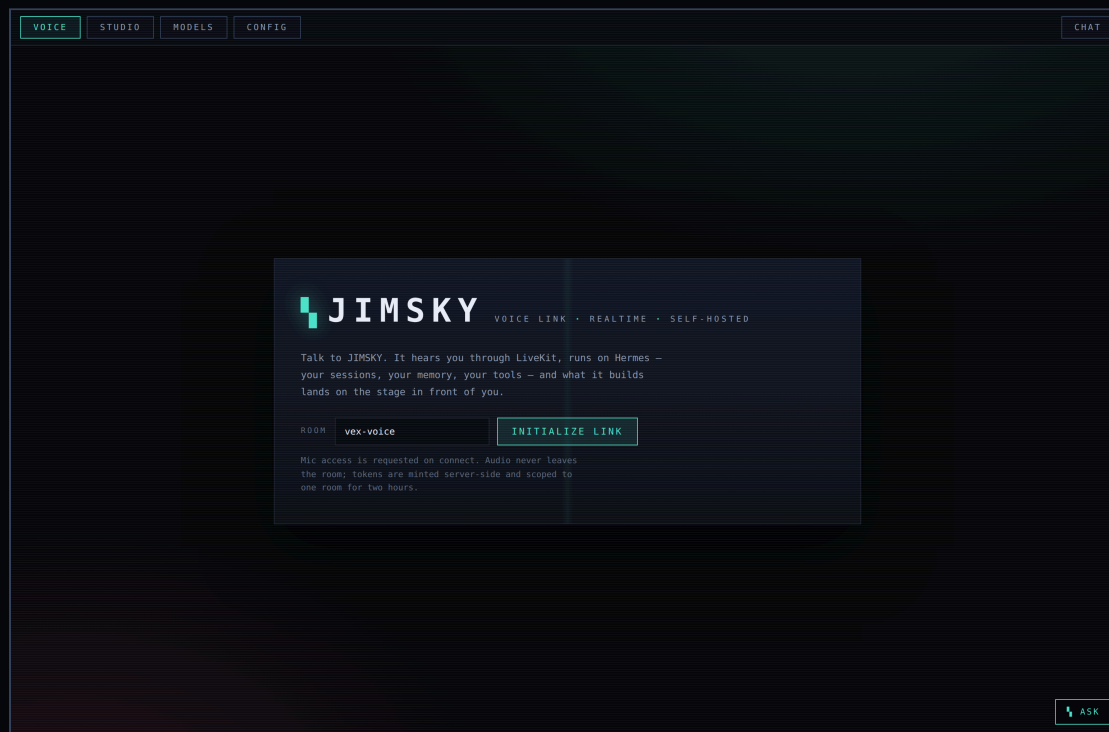
COMFY TEMPLATES

52

MODELS IN CATALOG

22

ROUTER MODELS



The voice room: talk to the agent over our own LiveKit server.

IN ONE LINE

A self-hosted voice + generation studio: a realtime voice agent you can talk to, a HUD that can make images and video, a model router that picks the brain per task, and a media stage that publishes the results — all on hardware we control, with approvals kept on the human side of anything that spends money or goes public.

NOTHING PUBLISHED

SOURCE ALPHA

NO VENDOR LOCK

Three halves that share one agent

THE VOICE

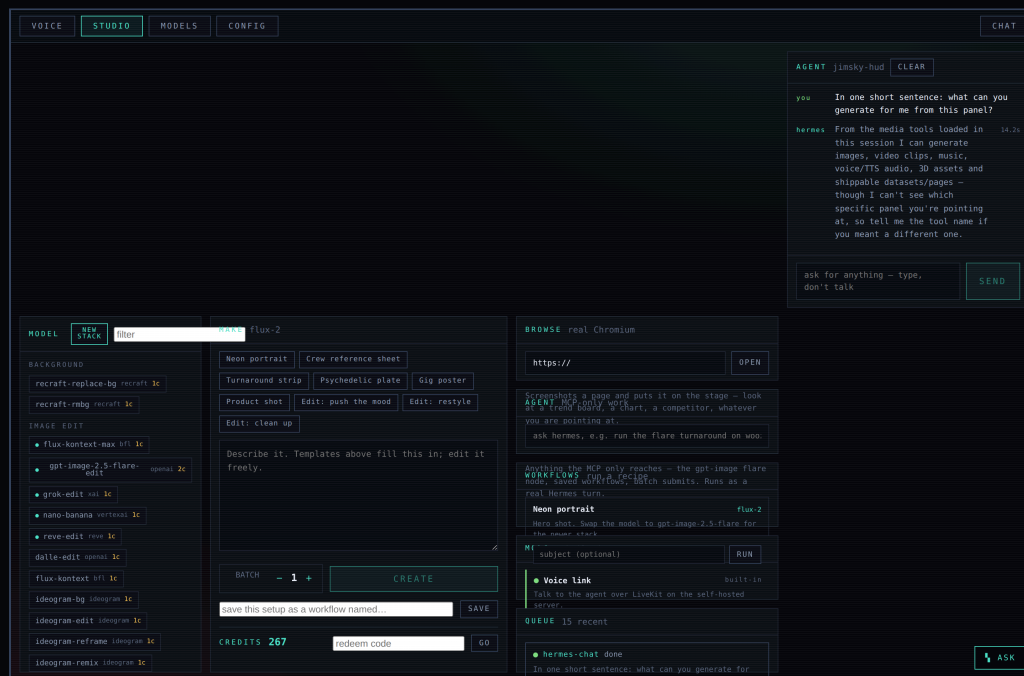
A realtime room on our own LiveKit server. You talk; the agent answers. Two engines are wired: a classic pipeline (speech-to-text → language model → speech) and **GPT-Live full-duplex**, which listens and speaks through the same model the way a person does. It hangs up on its own if nobody is talking — three layers of failsafe, so a forgotten tab cannot sit there burning a session.

THE MAKING

The HUD turns a sentence into an image, a batch, an edit, a browsed screenshot or a rendered video. It speaks to Comfy Cloud for generation and to a model router for deciding *which* brain does the work. 52 models across 13 categories, 629 official Comfy workflows browsable and executable, plus our own saved workflows.

THE STAGE

Anything made gets published to a media host we control, with a running index. The front end picks it up and puts it on screen — so the loop is: ask → make → see it, without leaving the page. Images, video and audio. This is what makes it a studio rather than a chat window.



The studio half: model picker, batch stepper, browse, credits, agent panel, mods and the job queue.

THE LOOP, AS IT ACTUALLY FEELS

Ask in the voice room → the agent reaches for the same tools a model can call → the HUD makes the thing → it is published to the stage → the front end puts it on screen while the conversation keeps going.

WHY A HUD AND NOT JUST AN APP

Because the interesting part is not one screen, it is having all of it in reach at once: talk, make, watch, and change the machine's settings without leaving the page you are working on.

The voice half stays mounted while you work in the studio, so a session does not have to be started and stopped to make something.

The stack, layer by layer

LAYER	WHAT IT IS	WHY / NOTE
Voice transport	LiveKit server, self-hosted (Apache-2.0) v1.13.5	our own VPS, TLS by Caddy — no hosting fee
Voice agent	worker vex , livekit-agents 1.8.1	registers and waits; it does not sit in a call
Two engines	pipeline · Deepgram STT → LLM → Cartesia TTS	gpt-live · full-duplex on gpt-live-1
Failsafe	180s idle · 30s empty room · 1800s cap	proven by a silent-probe test, agent left at ~66s
Front end	React + Vite → Netlify	jimsky-voice.netlify.app · phone + desktop
Token minting	Netlify Function, server-side	the LiveKit secret never reaches the browser
HUD backend	FastAPI on the VPS, X-HUD-Key gated	systemd user service, Linger on
Media stage	Caddy media dir + index.json	images, video and audio the agent publishes
Making images	Comfy Cloud + 52 models in 13 categories	gpt-image-2.5-flare stack, up to 2160×3840
Templates	629 official Comfy workflows	Comfy-Org/workflow_templates, executable from the HUD
Video	HyperFrames (HeyGen)	HTML composition → MP4, gated by lint + check
3D	Three.js	the orb in the HUD reflects agent state
Ledger	credit balance + redeem codes	one use each, kept on disk
Models	router across DeepSeek · OpenAI · OpenRouter	one prompt, several brains, measured

Upstream work is used under its own licence: LiveKit (Apache-2.0), ComfyUI and the Comfy-Org template library, HyperFrames (HeyGen), Three.js, Playwright, FastAPI. Our own code lives in private repos.

Pick the brain, not the app

The studio no longer has loyalty to one model. The router knows which brains we can reach and what each one is for, so a bulk job can run on a cheap flash model while judgement calls go to luna.

REACHABLE TODAY

- **Frontier** — GPT-5.6 luna, sol, GPT-5.5, DeepSeek V4 Pro. Luna is the voice agent's own backend, so the router and the room agree.
- **Fast + cheap** — DeepSeek Flash (Hermes' default), GLM 5.3 Flash, Qwen 3.8 Flash. This is where most calls should land.
- **Open weights** — Qwen 3.8 Max, DeepSeek V4 Flash Vision, GLM Flash. No vendor lock, and the honest test of whether closed models are still needed.
- **Free tier** — 22 OpenRouter models listed at \$0 per token. Testing costs nothing.
- **Self-hosted** — three RunPod Ollama endpoints, all currently answering 403. Reviving one pod puts open weights on our own silicon.

Every card shows its context window and its price per million tokens. Where a price is not published, the card says so instead of guessing.

VOICESTUDIOMODELSCONFIGCHAT

THE ROUTER

One agent, many brains. Pick up to four, give them the same prompt, and compare what actually came back – measured latency, token counts and cost. Nothing here is an estimate.

FRONTIER · the expensive brains. Luna is what the voice agent runs on.

GPT-5.6 LUNA

VOICE AGENT BRAIN

the model Vox runs on in GPT-Live full-duplex, our reference point.

openai 1050k ctx \$0.2/\$1.2 per Mtok

GPT-5.6 SOL

DEEP REASONING

heavier sibling of Luna.

openai 1050k ctx \$2/\$10 per Mtok

GPT-5.5

GENERAL

previous generation, still strong.

openai 1050k ctx \$5/\$30 per Mtok

DEEPSEEK V4 PRO

REASONING, CHEAP FRONTIER

near frontier output at a fraction of the price.

deepseek price unpublished

FAST + CHEAP · the everyday workhorses. this is where most calls should land.

DEEPSEEK FLASH

DEFAULT WORKHORSE

Hermes' own default brain. fast, cheap, good enough for most turns.

deepseek price unpublished

GLM 5.3 FLASH

BULK TASKS

1.3M context, the cron-pinned model - costs almost nothing per call.

openrouter 1311k ctx \$0.15/\$0.5 per Mtok

QWEN 3.8 FLASH

BULK + LONG CONTEXT

1M context, very cheap.

openrouter 1000k ctx \$0.15/\$0.47 per Mtok

OPEN WEIGHTS · no vendor lock. the honest test of whether we still need closed models.

QWEN 3.8 MAX

OPEN WEIGHTS, NEAR-FRONTIER

open-weights flagship, the honest test of 'do we still need closed models'.

openrouter 1000k ctx \$2/\$6 per Mtok

DEEPSEEK V4 FLASH VISION

OPEN WEIGHTS + VISION

sees images, useful for judging our own renders.

openrouter 1049k ctx \$0.22/\$0.66 per Mtok

GLM FLASH (latest)

OPEN WEIGHTS, CHEAPEST

1.3M context for pennies.

openrouter 1311k ctx \$0.073/\$0.23 per Mtok

FREE TIER · listed at \$0 per token, for testing without spending anything.

inclusionai/ling-3.0-flash-vl:free

ZERO-COST TESTING

OpenRouter lists it at \$0/token.

openrouter 202k ctx \$0/\$0 per Mtok

nex-agi/nex-n2.5-mini:free

ZERO-COST TESTING

OpenRouter lists it at \$0/token.

openrouter 202k ctx \$0/\$0 per Mtok

nex-agi/nex-n2.5-pro:free

ZERO-COST TESTING

OpenRouter lists it at \$0/token.

openrouter 202k ctx \$0/\$0 per Mtok

inclusionai/ling-3.0-flash-sante:free

ZERO-COST TESTING

OpenRouter lists it at \$0/token.

openrouter 202k ctx \$0/\$0 per Mtok

inclusionai/ling-3.0-flash-fin:free

ZERO-COST TESTING

OpenRouter lists it at \$0/token.

openrouter 202k ctx \$0/\$0 per Mtok

dots-studio/dots-3-note-preview:free

ZERO-COST TESTING

liquid/lfm-2.5-2.6b:free

ZERO-COST TESTING

OpenRouter lists it at \$0/token.

nvidia/nemotron-3.5-lightning:free

ZERO-COST TESTING

thinkingmachines/inkling-small:free

ZERO-COST TESTING

poolside/laguna-s-2.1:free

ZERO-COST TESTING

ASK

The router: tiers, roles, context and price on every card. Self-hosted status is shown honestly as offline.

Same prompt, three brains

“In exactly two sentences: what makes a rooftop at night feel alive? Be specific. No adjectives like beautiful.”

MODEL	PROVIDER	LATENCY	TOKENS IN → OUT	COST / CALL	NOTE
DEEPSEEK FLASH	deepseek	4891 ms	52→900	unpublished	price unpublished
GPT-5.6 LUNA	gpt	2155 ms	28→86	\$0.000109	fastest + cheapest
QWEN 3.8 MAX	qwen	10714 ms	84→393	\$0.002526	

Caught by the router, not hidden: DEEPSEEK FLASH — no visible answer came back — this model ran out of output budget inside its reasoning, so what follows is its thinking trace, not its answer. raise max tokens.

WHAT THE NUMBERS SAY

- **GPT-5.6 LUNA was fastest** at 2155 ms, and cheapest at \$0.000109 a call.
- **Slowest of the three: QWEN 3.8 MAX** at 10714 ms — 5.0× the fastest.
- **Open weights are a freedom move, not automatically a savings move** — the open-weights entry here costs \$0.002526, 23× the cheapest model in this run.
- **1 of 3 models have no published price** in our source, so any total is a floor rather than a fact.
- All three ran **in parallel**; wall clock 10717 ms, total \$0.002635.

The router reports what came back, not what it hoped for: a truncated answer is flagged, and if a model spends its whole budget on hidden reasoning the panel says the text below is a thinking trace rather than an answer.

3 models ran in parallel - wall clock 10717 ms - total \$0.002635 (some prices are unpublished, so this total is a floor)
fastest: GPT-5.6 LUNA at 2155 ms - cheapest: GPT-5.6 LUNA at \$0.000109 per call

The verdict line, computed from the run itself.

Three brains, one prompt, one wall clock. Each panel carries its own latency, token count, cost and the source of that price.

- Each answer names its **provider and exact model id**, so nothing is ambiguous later.
- Latency is the wall clock for that call; tokens come from the provider's own usage block.
- Cost is computed only from a published price, and the **source of that price is printed** under every panel.
- **TRUNCATED at cap** appears when a model hit the output ceiling — visible, not buried.
- **from reasoning field** appears when a model produced no visible answer and only its thinking came back.

Because "which model is better" is usually answered with opinions. Here it is answered with the numbers that came back, from the same run, with the caveats left in where they belong — next to the claim.

Open-source models, and an agent that can drive the whole machine

WHERE OPEN MODELS STAND

Open weights are **already reachable** through the router — 137 candidates on one provider, 22 of them free, including vision models. What is missing is the fully private end: a model running on hardware we own, with no vendor in the path at all.

Blocked, and it is honest: the three RunPod Ollama endpoints answer 403. The pods are down. Bringing one up costs roughly \$0.5/hr and would give us a private model — worth doing the moment a task genuinely needs privacy rather than price.

WHY THE ROUTER MATTERS FOR THIS

Because the *mind* is swappable and the *hands* are not. Every model here drives the same tools, so moving work to an open model later is a selection, not a rewrite.

WHAT THE AGENT CAN ALREADY DRIVE

REACH	STATE
Voice room	live — talk to it in real time
Browser	live — a real Chromium it loads, reads and screenshots
Machine	live — shell, files, services, git, cron
Make	live — images, video, batches, 629 templates
Publish	live — writes to the stage this front end reads
Delegate	live — sub-agents and background jobs
Desktop GUI	not wired — a mouse and keyboard, not just a shell

The desktop gap is the difference between "it can run a command" and "it can operate any program the way you would". That is the next real capability, and it is a wiring job, not a research problem.

What is proven, what is not, and what is missing

LIVE AND VERIFIED

- self-hosted LiveKit + the vex worker, both engines, GPT-Live confirmed in the log
- front end deployed and talking; token minting verified in production
- idle failsafe proven at both ends (agent left after ~66s of silence while the human stayed)
- HUD studio: 52 models, 629 templates, batch runs, browse, chat, credit ledger
- model router: real completions, measured latency and tokens, prices with their source attached
- the ten-voice video: rendered, gate-passed, visually verified, published to the stage

BUILT BUT NOT PROVEN

- local ComfyUI render on the 4070 — the box does not answer on the tailnet address
- the React-Native mobile app compiles (tsc clean) but has never run on a device
- LiveAvatar face mod is registered and waiting on a paid third-party key

HONEST GAPS

- RunPod Ollama proxies answer 403 — self-hosted open weights are offline right now
- desktop GUI control is not wired; the agent drives a browser, a shell and files, not a mouse
- DeepSeek does not publish per-token prices in our source, so its cost column reads unpublished
- hosted web search on the backend model is not enabled



The same router on a phone — the format this is actually meant to live in.

HOW IT IS MEANT TO BE USED

As a small HUD: on a phone in the kitchen, on a tablet on a stand, or on a desktop while working. The voice half stays mounted while you make things, so you can talk to it and watch the stage fill up at the same time.

MONEY AND SAFETY POSTURE

Every backend call needs a key, because these endpoints can spend money. Voice sessions die on their own. Anything that spends real money, goes public, or touches infrastructure stays on the human side of an approval, and the router reports cost per call so the bill is never a surprise.

SELF-HOSTED VOICE

KEY-GATED BACKEND

SPEND IS VISIBLE

Surfaces, services and repos

SURFACES

Studio front end	jimsky-voice.netlify.app
Media stage	jimsky-media...nip.io
HUD backend	jimsky-hud...nip.io
Voice server	vex...nip.io

SERVICES ON THE VPS

LiveKit	docker voice-livekit-1 · 7880/7881
TLS front	docker voice-caddy-1
Voice worker	vex-agent.service
HUD backend	jimsky-hud.service · 8099

WHAT IS WHERE

- **Voice** — apps/voice: the compose stack, the LiveKit config, the agent.
- **Front end** — jimsky-voice/web: the screens, the token function, the continuation file.
- **HUD** — jimsky-hud: the FastAPI backend, the Comfy runner, the router.
- **Voice lab** — voice-lab: ten voices as entities, the design conversation, the video project.
- **Secrets** — profile .env only, mode 600, referenced by name, never printed.

THE LOOP, END TO END

ask in the voice room → the agent uses the same tools a model can call → the HUD makes the thing → it is published to the stage → the front end shows it while the conversation continues.